

华北山区农作物的种植策略研究

摘要

种植业是经济社会发展的重要支柱，近年来自然及人为因素导致耕地数量、质量下降，农作物生产问题加剧，随人口增加，农作物需求变大，引发一系列经济、生态问题。为响应科技兴农，因地制宜，科学提高土地利用效率，保证耕地安全的前提下增加农作物产量与总经济效益，本文利用线性规划、随机森林模型与最坏适应算法，探索最优种植模式，应对种植风险，保障经济、社会、生态可持续发展。

针对问题一的第一问，第一步进行数据预处理，计算出各类农作物在不同类型土地的每亩经济效益和不同类型土地的可种植亩数，并对统计数据进行可视化处理，分析得到不同作物在各土地类型面积分布、单位经济效益方面有显著性差异。第二步，以总经济效益为目标函数，设计土地资源限制和农作物产量限制作为约束条件，构建线性规划模型。利用 **Linprog** 函数求解初步得到各类农作物在每种类型土地的总种植亩数。第三步，引入植物种类分配最坏适应算法，在题目给出的约束条件下优化算法得到农作物在具体地块的种植情况。第二问在第一问结果的基础上在转换部分约束条件，按照第一问的思路得到新的目标函数，再次利用植物种类分配最坏适应算法，即可得到问题一第二问的种植策略。

针对问题二，第一步在该问题中农作物的预期销售量、亩产量、种植成本和销售价格等因素均可影响到土地的最终经济效益。引入随机森林模型识别经济效益的关键性因子，分析各因素重要性排行，得到销售单价和亩产量为优化种植问题中的关键因子。第二步，用 **Excel** 处理题目中预测的变化量，经随机森林模型识别出销售单价与亩产量为总经济效益波动的关键因子，为后续因对因子变化的分析中提供重要性排行。第三步，我们在问题一的基础上，优先考虑两个关键因子对线性规划模型的影响，简化最坏适应算法，得到问题二的种植策略。

对于问题三，我们结合生产生活实际与相关专业知知识，从作物间可替代性、不同作物间的相互影响、农作物对土地的改良、生产与销售间的供需关系及我国的国家政策等方面给出具体的优化种植方案，探索出可持续发展道路。

最后，本文对所建立的模型进行了综合优缺点分析与推广。

关键字： 线性规划模型 最坏适应算法 随机森林模型 优化模型算法

一、问题重述

1.1 研究背景

种植业是农作物生产的主要来源，对经济发展、社会稳定具有重要意义。近年来随人口快速增长，城市化水平的不断提高、全球气候显著变化，耕地质量下降，我国农作物生产供不应求态势日益凸显。[1] 华北地区位于中纬度地区，水热条件及耕地复种指数相对较低，而华北地区经济发达人口稠密，故此人地矛盾问题亟待解决。随科技发展，温室大棚等农业设施助力科技兴农，因地制宜、轮作套种、调整农作物种植结构等合理利用耕地的方法可有效提高当地居民收入的同时，保障耕地的可持续利用，提高土地利用效率，对乡村振兴、保障国家粮食安全意义重大。因此本文运用线性规划、随机森林等模型探究华北山区乡村最优种植模式有望为土地利用政策的制定提供可靠依据，保障我国耕地安全与可持续发展。

1.2 问题要求

本文着手分析 2023 年农作物及土地类型数据，运用数学思想，建立线性规划、最坏适应算法、随机森林模型来解决耕地分配中的以下问题：

（1）问题一：情况一，超过部分滞销而造成浪费；情况二，超过部分按 2023 年销售价格的 50% 降价出售，分别探讨在两种情况下，2024-2030 年间各年份最佳土地分配与农作物种植方案。

（2）问题二：在气候变化、市场波动和预期销售量等因素波动的条件下，优化 2024-2030 年农作物种植方案，实现收益最大化。

（3）问题三：进一步探究各因子间的相互作用，结合互补性，考虑相关性，基于实际生产情况，进一步优化 2024-2030 年种植方案，提升整体收益和抗风险能力。

二、问题分析

本题主要分为，基于往年数据及限制条件的线性规划模型，解决农作物和土地的分配问题，并利用优化算法进行资源的合理分配。同时，通过主控因子的分析，揭示问题中的关键影响因素。对题目分析发现，在土地分配过程中，数据零散且限制性条件较多，各条件间相互关联，故应首先将数据进行预处理整合，以理清变量间关系，更直观把握数据，保障后文模型准确性与直观性。

2.1 问题一分析

针对问题一，第一小问，欲基于条件中所给的预期销售量、种植成本、亩产量、销售价格等信息，得到在超过部分滞销情况下的最优耕地分配，则需尽量减少超产部分避免粮食浪费，使得 2023-2030 年的产量与期望产量（即 2023 年的产量）尽量接近。我们采用线性规划方法，以在满足约束条件下求出目标函数的最大值，即使得得到总经济收益最大。再通过最坏适应算法实现土地分配，得到具体的最优种植方案。

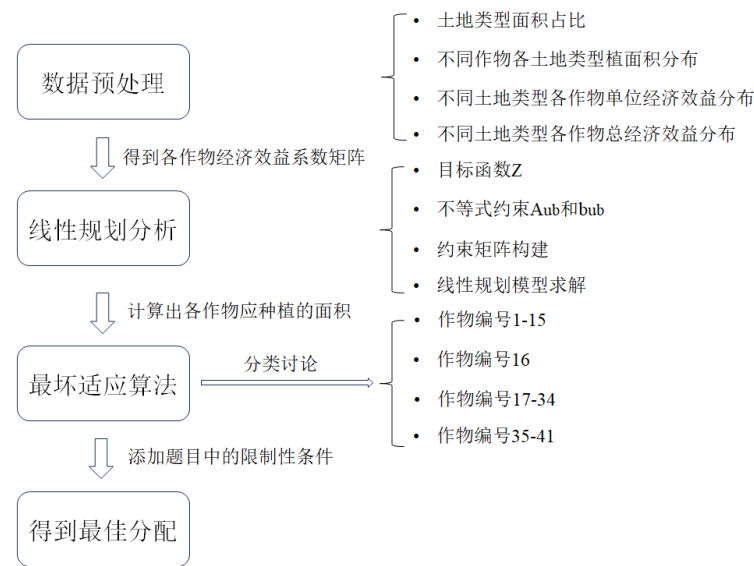


图 1 问题一流程图

2.2 问题二分析

通过问题一，我们能够得到一个目标函数与一个约束条件的集合，即本问题中所提到的预期销售量，种植成本，亩产量，销售价格等影响因子，均被问题一中所提出的目标函数所包含。因此，我们仅仅需要按照题目中所设的内容，对各个影响因子进行合理的调整，例如对每个因子乘以一个变化率，此时便可以得到一个新的目标函数与约束条件，通过对该目标函数进行求解，能够得到一个具体的最优解决方案。

2.3 问题三分析

对于问题三，重点在于结合生产生活实际，考虑题目中所给出的特殊限制条件与影响因素，给出具体的优化种植方案，探索出生态、经济、社会可持续发展道路。

三、模型的假设

- 假设预期销售量与 2023 年生产量相同。
- 假设所提供的所有其他条件均相同。

- 忽略天气和植物自身生长条件等其他不确定因素对最终农作物产量和销量的影响。
- 确信附件中所提供的数据均真实可靠且有实际参考价值。

四、符号说明

符号	意义
$ABCDEF$	土地 (乡村耕地和大棚) 类型
Z	总经济收益
x_{iY}	编号为 i 的农作物在 Y 类型土地的种植亩数
C	各类农作物在各类土地上每亩平均经济效益矩阵
c_{ij}	编号为 i 的农作物在 j 类型土地的每亩经济效益
A_{ub}	不等约束矩阵
a_{ij}	编号为 i 的农作物在 j 类型土地的每年亩产量
b_{ub}	2023 年各类农作物的每年总产量

五、模型的建立与求解

5.1 问题一模型的建立及求解

5.1.1 模型的准备：数据可视化与处理

附件 2 中给出了 2023 每种农作物的种植亩数、单价、成本等，可以计算得到每种农作物每亩的单价。为避免农产品滞销造成浪费，需要使 2024-2030 年的农作物产量尽量与 2023 年接近，因此也需要计算得到 2023 年不同农作物的总经济收益。

1. 土地类型面积占比

题目中附件 1 中“乡村的现有耕地”表格中可知，研究区域华北山区有六种土地类型，各土地类型又被细分为各个地块，因此统计较为零碎，不利于后续计算，本研究将不同地块按土地类型进行分类，加和计算出各土地类型的面积，运用 R 语言画出饼状图，将土地类型面积占比进行可视化。由图 2 可知，不同土地类型面积差异显著，其中梯田面积占比最大，达总面积的 51.03%，平旱地次之，占比 30.09%。大棚类占比最低，普通大棚和智慧大棚分别占总面积的 0.79% 和 0.20%。

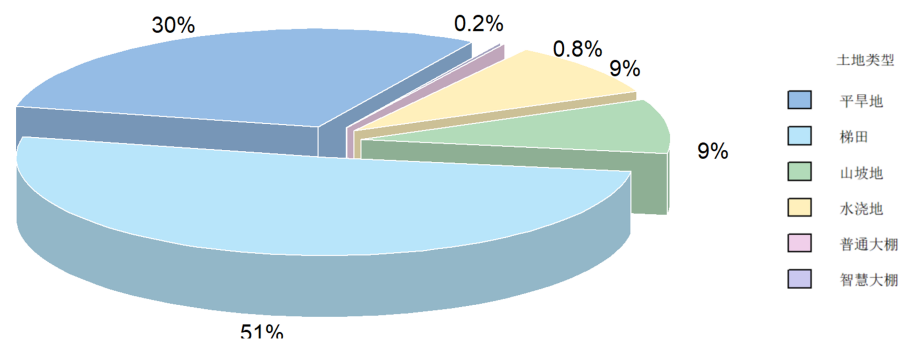


图2 土地类型面积占比

2. 不同作物各土地类型种植面积分布

依据题目中附件2中“2023年农作物种植情况”可知不同作物在各地块上的分类，与1类似，农作物数量以及地块种类众多导致数据庞杂，因此先将同种作物在同种土地利用类型上的不同地块面积进行加和，得到各作物在不同土地利用类型上的分布。由图3可知，黄豆、绿豆、玉米等旱作粮食作物主要分布于平旱地，而黑豆、红豆、小麦、谷子、莜麦、大麦等粮食作物主要分布于梯田，荞麦、南瓜、红薯以山坡地种植为主。水稻、豇豆等以水浇地为主，其他各经济作物以大棚种植为主。

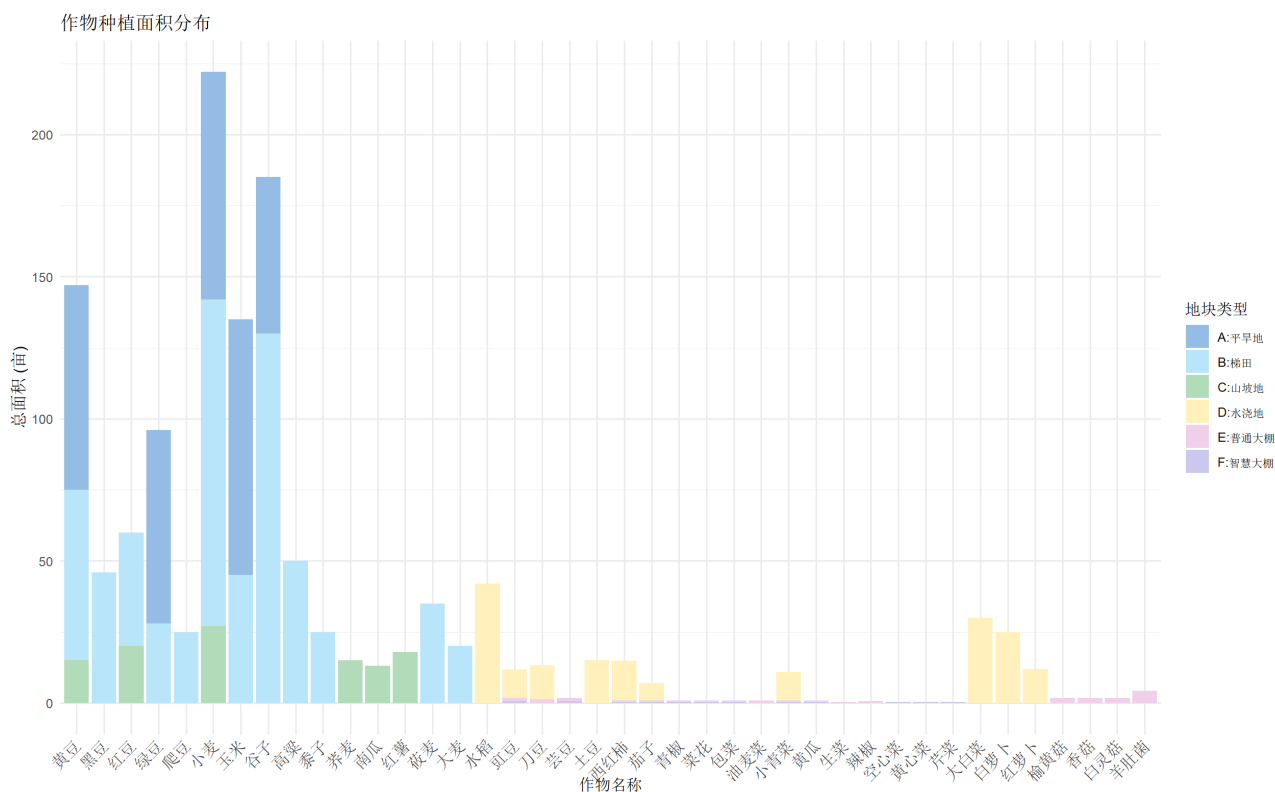


图3 不同作物各土地类型种植面积分布

3. 不同土地类型各作物单位经济效益分布

附件 2 中“2023 年统计的相关数据”中提供了不同作物在各土地利用类型上的种植成本、销售单价，此两种数据对于后续线性规划模型设计并无直接意义，因此首先用公式：每种农作物每亩单价 = (亩产量 × 销售单价) - 种植成本；计算出每种作物在各土地利用类型上的单位经济效益。由图 4 可知，蔬菜及食用菌等经济作物单位经济效益值整体显著高于粮食作物，大棚的单位经济收益也整体大于各类型耕地。大棚内部也有显著差异，智慧大棚单位经济效益整体高于普通大棚。

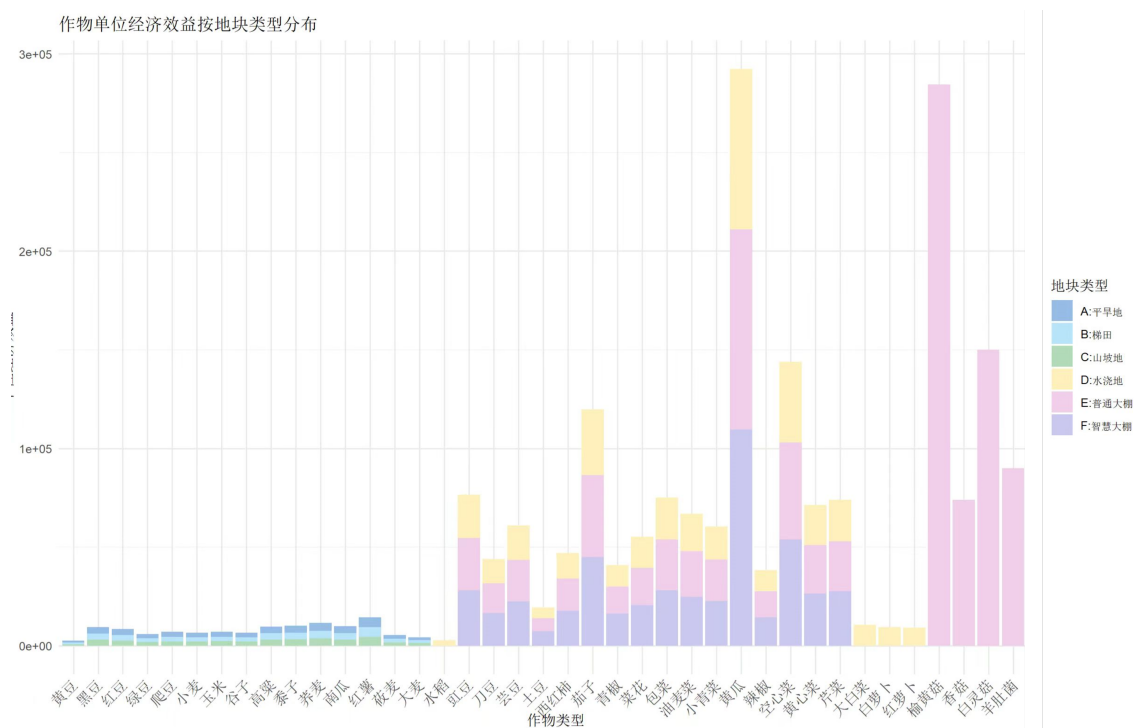


图 4 不同土地类型各作物单位经济效益分布

4. 不同土地类型各作物总经济效益分布

在 3 的基础上, 作物总经济效益 c_{ij} 可由公式 $c_{ij} = \sum (\text{农作物对应土地类型} \times \text{种植亩数})$ 得到, 由图 5 可知, 与 3 显著不同, 作物总经济效益还受到种植量的显著影响小麦、玉米、谷子、大白菜、萝卜等作物由于种植面积大而总经济效益高。

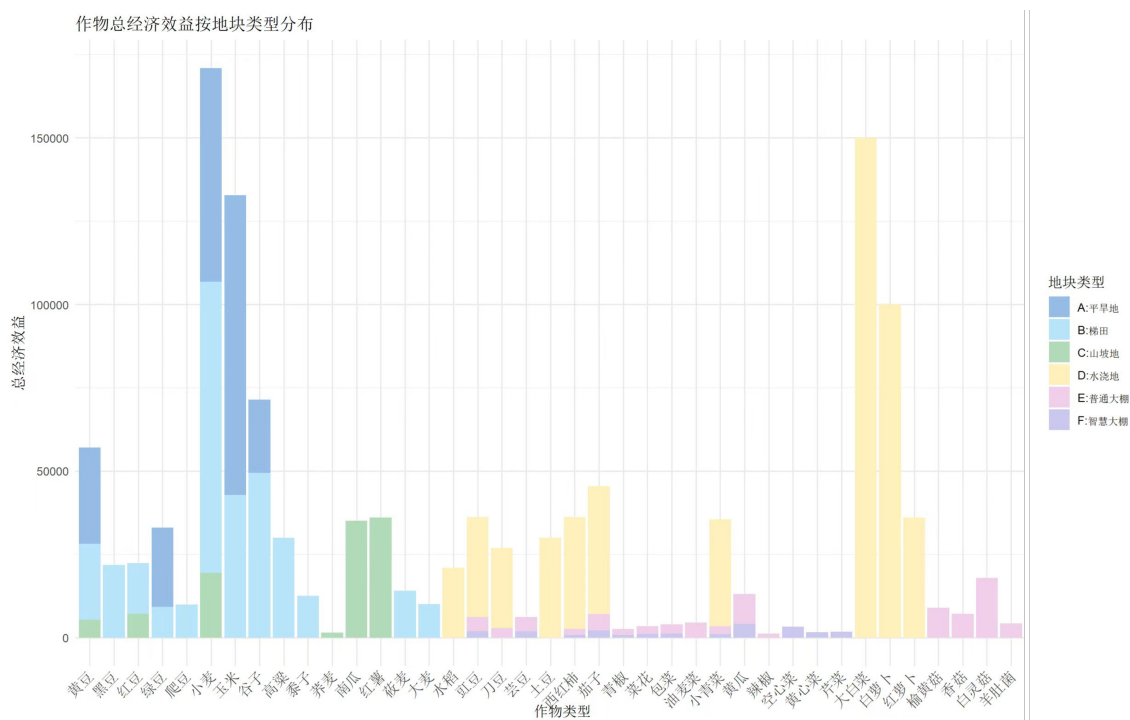


图 5 不同土地类型各作物总经济效益分布

5.1.2 线性规划模型

线性规划模型是用于优化某一目标函数的数学方法，通常在资源分配问题中用于最大化或最小化线性目标函数。本研究中，目标函数总经济收益 Z 与各类农作物在各类型土地上的每亩的平均经济效益 x_{iY} 呈现线性关系，故可采用线性规划模型优化出每种农作物的最佳种植面积。

决策函数每种作物总种植面积为得到超过部分滞销情况下农作物在不同土地利用上的总体最优分布，本研究以总经济收益 Z 为目标函数，每种农作物种植亩数 x_{iY} 为决策变量，得到线性规划方程

1. 目标函数 Z

目标函数设计：

$$Z = \mathbf{C}\mathbf{X} \quad (1)$$

$$\begin{aligned}
Z = & \left(\sum_{i=1}^{15} (x_{iA} \cdot c_{i1} + x_{iB} \cdot c_{i2} + x_{iC} \cdot c_{i3}) \right) + (x_{16D} \cdot c_{16,1}) \\
& + \left(\sum_{j=17}^{34} (x_{jD_1} \cdot c_{j1} + x_{jE_1} \cdot c_{j2} + x_{jF_2} \cdot c_{j3}) \right) \\
& + \left(\sum_{k=35}^{37} x_{kD_2} \cdot c_{k1} \right) + \left(\sum_{l=38}^{41} x_{lE_2} \cdot c_{l1} \right)
\end{aligned} \tag{2}$$

$$\mathbf{X} = \begin{bmatrix} x_{1A} & x_{1B} & x_{1C} & \cdots & x_{41E_2} \end{bmatrix}^T$$

$$\mathbf{C} = \begin{pmatrix} c_{1,1} & \cdots & c_{1,3} \\ \vdots & \ddots & \vdots \\ c_{41,1} & \cdots & c_{41,3} \end{pmatrix}$$

(1) 式中, Z 为目标函数, 代表了总经济收益; X 为决策变量, 代表了不同编号的农作物在不同土地种植的亩数; C 为价值向量矩阵, 代表 2023 年对应农作物在特定土地每亩的经济效益;

(2) 式中的决策变量划分:

在模型中, 我们通过设置 107 个决策变量 X_i , 每个决策变量代表某类作物在某类土地上的种植亩数。这些决策变量乘以对应的经济效益系数 C_i 后, 即可得到当年的总经济收益。具体划分如下:

(a) 1-15 号农作物

- 只能种植在 $A/B/C$ 类型的土地上。
- 决策变量 $X_{1A}, X_{1B}, X_{1C}, \dots, X_{15A}, X_{15B}, X_{15C}$ 代表作物编号为 1-15 的农作物在不同土地类型上的种植面积。

(b) 16 号农作物

- 适宜种植在 D 类型的土地上。
- 对应的决策变量为 X_{16D} , 表示 16 号作物在 D 类型土地上的种植亩数。

(c) 17-34 号农作物

- 适宜种植在 $D_1/E_1/F_2$ 类型的土地上。
- 决策变量为 $X_{17D_1}, X_{17E_1}, X_{17F_2}, \dots, X_{34D_1}, X_{34E_1}, X_{34F_2}$ 。

(d) 35-37 号农作物

- 适宜种植在 D_2 类型的土地上。
- 决策变量为 $X_{35D_2}, X_{36D_2}, X_{37D_2}$ 。

(e) **38-41** 号农作物

- 适宜种植在 E_2 类型的土地上。
- 对应的决策变量为 $X_{38E_2}, X_{39E_2}, X_{40E_2}, X_{41E_2}$ 。

(2) 式为 (1) 式的具体展开形式，结合具体说明如下：

$i \in [1, 15]$ 时，代表的是作物编号为 1 – 15 的农作物分别在

$A/B/C$ 类型土地的种植亩数乘以 $A/B/C$ 类型土地上的经济效益；

$i = 16$ 时，代表的是作物编号为 16 的作物在 D 类型土地的种植亩数乘以

D 类型土地上的经济效益；

$i \in [17, 34]$ 时，代表的是作物编号为 17 – 34 的农作物分别在

$D_1/E_1/F_2$ 类型土地的种植亩数乘以 $D_1/E_1/F_2$ 类型土地上的经济效益。

为了保证模型的通用性和可操作性，我们通过增加决策变量的数量，使得每个农作物与其适宜的土地类型对应。通过设置上述参数设置，确保了每类农作物的种植仅限于其适宜的土地类型上，同时避免了不合理的土地利用。

这样避免了在模型构建中逐一考虑作物和土地之间的匹配关系，简化了算法的设计与实现。所有的农作物和土地类型组合都已在决策变量中明确表达，因此无需额外进行组合优化。

2. 不等式约束 A_{ub} 和 b_{ub}

约束条件设计：

在构建线性规划模型的过程中，除了目标函数的优化外，约束条件的设置至关重要。

本文根据题干条件与各参数之间的约束关系通过寻找合适的 $\leq$ 不等式约束条件，确保农作物在不同土地类型上的种植面积满足实际的资源限制。该题目中约束条件主要来源于两个部分：土地资源限制和农作物产量限制。具体设计思路如下：

2.1 土地资源限制

首先，我们根据 5.1.1 中土地资源可视化中的数据可以知道各类型土地的分布情况，从附件 1 中的 `sheet1` “乡村的现有耕地”中提取了不同类型土地种植面积的限制条件。并且通过 `python` 对每一行数据的系数进行读取并映射到约束条件的右侧即 b_{ub}

同时我们在 `excel` 中构建了 41×3 阶矩阵，从矩阵中寻找到不同类型土地所对应的可种植农作物亩（即决策变量），同一种类型土地所对应的决策变量之和要小于每

一大类土地总的可种植面积。

此时，构建了一个约束条件矩阵 A_{ub} ，该矩阵的每一行代表了某一类农作物在某一类土地上的种植面积限制。这些约束条件可以通过以下方式进行划分：

- (a) 第 1 至 15 号农作物：这些农作物只能种植在 $A/B/C$ 类型的土地上。我们为每种土地类型设置了约束条件，确保各作物在这三类土地上的种植总面积不超过给定的上限。

$$\begin{aligned}\sum_{i=1}^{15} x_{iA} &\leq A\text{类型土地总亩数} = 365 \\ \sum_{i=1}^{15} x_{iB} &\leq B\text{类型土地总亩数} = 619 \\ \sum_{i=1}^{15} x_{iC} &\leq C\text{类型土地总亩数} = 108\end{aligned}$$

- (b) 第 16 号农作物：只能种植在 D 类型的土地上，对应的约束条件限制了该作物在该土地类型上的种植面积。
- (c) 第 17 至 34 号农作物：这些作物适宜种植在 $D_1/E_1/F_2$ 类型的土地上，我们分别为这三种土地设置了相应的面积限制。
- (d) 第 35 至 37 号农作物：仅适合种植在 D_2 类型土地上，对应的种植面积受到相应约束。
- (e) 第 38 至 41 号农作物：适宜种植在 E_2 类型土地上，也有相应的种植面积上限。

$$\sum_{i=17}^{34} x_{iD_1} + \sum_{j=35}^{37} x_{jD_2} + x_{16D} \leq D\text{类型土地总亩数} = 109$$

$$\sum_{i=17}^{34} x_{iE_1} + \sum_{j=38}^{41} x_{jE_2} \leq E\text{类型土地总亩数} = 9.6$$

$$\sum_{i=17}^{34} x_{iF_2} \leq F\text{类型土地总亩数} = 2.4$$

通过设置上述约束条件，确保了每类农作物的种植面积小于所题目中所给定的各类土地面积。

2.2 农作物产量面积限制

为了进一步确保每类作物的种植面积不超出既定 2023 年的生产规划，本研究添加了额外的种植面积约束条件。这些约束基于农作物的总种植面积要求，具体设计如下：

- (a) 对于第 1-15 号农作物，我们分别对每种作物在 $A/B/C$ 类型土地上的亩数与亩产量之积的和进行了限制，确保 1-15 号农作物的总产量不超过 2023 年的总产量。相应的数学表达式为：

$$\sum_{i=1}^{15} (x_{iA} \cdot y_{iA} + x_{iB} \cdot y_{iB} + x_{iC} \cdot y_{iC}) \leq \text{2023 年 } i \text{ 号农作物总产量}$$

其中, x_{iA}, x_{iB}, x_{iC} 分别表示第 i 号农作物在 $A/B/C$ 类型土地上的种植面积, y_{iA}, y_{iB}, y_{iC} 为对应的 2023 年单位亩产量。

- (b) 第 16 号作物在其适宜土地类型 D 上的种植亩数与亩产量之积限制为不超过 2023 年 16 号农作物的总产量:

$$x_{16D} \cdot y_{16D} \leq \text{2023 年 16 号农作物总产量}$$

- (c) 针对第 17-34 号农作物, 我们分别对每种作物在 $D_1/E_1/F_2$ 类型土地上的亩数与亩产量之积的和进行了限制, 确保 17-34 号农作物的总产量不超过 2023 年的总产量。对应的数学表达式为:

$$\sum_{i=17}^{34} (x_{iD_1} \cdot y_{iD_1} + x_{iE_1} \cdot y_{iE_1} + x_{iF_2} \cdot y_{iF_2}) \leq \text{2023 年 } i \text{ 号农作物 2023 年总产量}$$

- (d) 针对第 35-37 号农作物, 我们分别对每种作物在 D_2 类型土地上的亩数与亩产量之积的和进行了限制, 确保 35-37 号农作物的总产量不超过 2023 年的总产量:

$$\sum_{i=35}^{37} x_{iD_2} \cdot y_{iD_2} \leq i \text{ 号农作物 2023 年总产量}$$

- (e) 针对第 38-41 号农作物, 我们分别对每种作物在 E_2 类型土地上的亩数与亩产量之积的和进行了限制, 确保 38-41 号农作物的总产量不超过 2023 年的总产量:

$$\sum_{i=38}^{41} x_{iE_2} \cdot y_{iE_2} \leq i \text{ 号农作物 2023 年总产量}$$

这些限制条件的设计旨在确保农作物种植的合理性与有效性, 避免农作物滞销, 造成资源浪费和土地的过度使用。

通过对约束矩阵的构建和扩展, 能够全面反映农作物在不同土地类型上的种植面积限制。这种设计保证了模型的可操作性与通用性, 同时极大简化了实际算法的实现。

3. 约束矩阵构建的总体思路

约束矩阵

为了便于模型的计算和实现, 我们将上述所提到的两大类所涉及到的所有约束条件统一表达为线性不等式, 构建了约束矩阵 A_{ub}

矩阵的每一行对应一个约束条件, 矩阵的每一列对应一个决策变量。通过对约束矩阵的构建和扩展, 能够全面反映农作物在不同土地类型上的种植面积限制。这种设计保证了模型的可操作性与通用性, 同时极大简化了实际算法的实现。

3.1 初始化不等式约束矩阵:

定义一个 47×123 的矩阵 A_{ub} ，用于存储不等式约束系数：

$$A_{ub} = \begin{pmatrix} a_{11} & a_{12} & \dots & a_{1,123} \\ a_{21} & a_{22} & \dots & a_{2,123} \\ \vdots & \vdots & \ddots & \vdots \\ a_{47,1} & a_{47,2} & \dots & a_{47,123} \end{pmatrix}, \quad A_{ub} \in \mathbb{R}^{47 \times 123} \quad (3)$$

(3) 式中, A_{ub} 为约束矩阵, 前 41 行对应连了 2.2 中的 41 个产量约束条, 第 42-47 对应了 2.1 中的 6 个土地约束; a_{ij} 约束矩阵系数, 代表了土地约束或者产量约束中的系数;

为了满足后续求解 *linprog* 的方法, 此处我们扩充先前与模型说明中的 107 个参数对应的系数个数至 123 个以保证矩阵的完整性, 补充的系数为 a_i , $i = 46, 47, 48, 103 + 3k, 104 + 3k (k \in [0, 6])$

3.2 填充约束条件矩阵:

对于每个约束 ($i = 1, \dots, 47$), 从整合后的 **excel** 数据表 (含有土地约束和产量约束中的系数) 中读取当前行的前三个系数, 并将其放入矩阵 A_{ub} 的对应位置。假设第 i 行的系数为 $\text{row} = [c_{i1}, c_{i2}, c_{i3}]$, 则我们有:

$$A_{ub}(i, [3i - 2, 3i - 1, 3i]) = [c_{i1}, c_{i2}, c_{i3}]$$

其中, $3i - 2, 3i - 1, 3i$ 分别表示与第 i 行对应的三个决策变量索引。

3.3 不等式约束的右侧系数:

从整合后的 **excel** 数据表 (含有不同编号产物每年总共销售产量和不同类型土地总亩数) 中读取不等式约束的右侧数值向量 b_{ub} , 该向量对应每个约束的不等式右侧系数:

$$b_{ub} = \begin{pmatrix} b_1 \\ b_2 \\ \vdots \\ b_{47} \end{pmatrix}, \quad b_{ub} \in \mathbb{R}^{47}$$

3.4 线性不等式系统的表达:

最终, 整个不等式约束系统可以表示为如下矩阵形式: [2]

$$A_{ub} \cdot \mathbf{X} \leq b_{ub}$$

其中, $\mathbf{x}$ 是包含 123 个决策变量的向量:

$$\mathbf{x} = \begin{pmatrix} x_1 \\ x_2 \\ \vdots \\ x_{123} \end{pmatrix}, \quad \mathbf{x} \in \mathbb{R}^{123}$$

4. 线性规划模型求解

模型建立：

综上可以得到农作物土地种植面积的线性规划模型：

目标函数：

$$\text{Minimize: } Z = -C^T X \quad (4)$$

约束条件：

$$s.t. \begin{cases} A_{ub} \cdot X \leq b_{ub} \\ x_i \geq 0, \quad i = 1, 2, \dots, 123 \end{cases}$$

模型求解：

提供的线性规划问题中，多个求解器都返回了无最优解的结果，并提示问题是无界的。

可行区域和约束：

根据图像，可以看到不同的约束条件定义了一个可行区域（feasible region）。

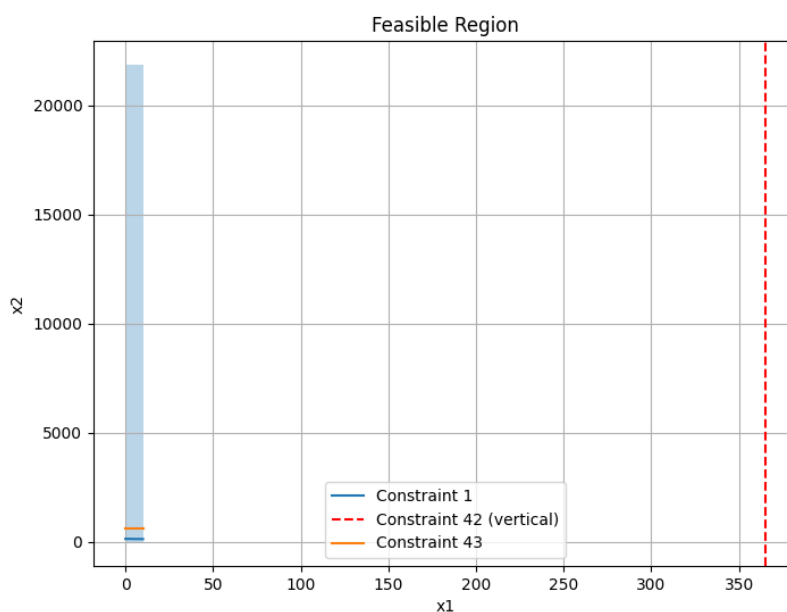


图 6 Linprog

结论:

在代码中, 多个求解器返回了无解的结果, 并提示问题是无界的。模型无界, 无法找到有限的最优解, 这种情况与可行区域的无界性相一致。

5.1.3 植物种类分配的最坏适应算法

由上述得到的线性规划方程无解可以得到, 在第一小问题的约束条件和情况下, 该线性规划问题不存在最优解, 因此我们将 23 年的种植情况视作目前的最优解。

为解决植物种类分配问题, 我们在此处引入最坏适应算法。最坏适应算法 (Worst-Fit Algorithm) 是来源于计算机领域中的一种算法, 旨在连续地进行动态内存分配。它是一种用于资源分配问题的算法, 它通常用于内存分配和作业调度等场景。其核心思想是将资源分配给当前最“差”的或最不合适的部分, 以尽量避免未来的资源浪费。优点是: 可使剩下的空闲分区不至于太小, 产生碎片的几率最小, 对中、小作业有利, 同时该算法查找效率很高。

考虑到本研究中的情景与该算法背景有相似之处, 需求是尽可能少的产生较小的土地块, 因此, 我们考虑在此处引入最坏适应算法。

因此针对问题一的第一小问, 我们综合统计出 2023 年每一种农作物在其对应类型土地种植的总亩数, 将其用作 2024 年植物种类分配的分配目标, 由此即可得到 2024 年的最佳植物种植方案。而后续年份的植物种类分配, 只需要对最坏适应算法中的目标种植方案进行迭代, 即可得到未来一年种植方案。

最坏适应算法的数学公式为:

$$W(n) = \max_{i=1}^n \{M[i]\}$$

其中, $W(n)$ 代表最坏适应分配算法的分配结果, $M[i]$ 代表内存空间大小 [3]

在已确定不同作物种植总面积的基础上, 为设置各作物类型在不同土地利用类型的分布, 且保障每种作物不在同一地上连茬种植, 所有土地三年内至少种植一次豆类作物、同种作物种植地尽量集中等条件, 本研究采用最坏适应算法, 每次分配均选择当前最大的空闲内存块来进行分配, 选择最大空闲块后, 就会将所需的内存从该块中划分出来, 并将剩余的部分继续作为空闲块保留在系统中。

具体实现步骤参考下面的伪代码:

Algorithm 1 植物种类分配的最坏适应算法（第 1 部分）

初始化: 1. 初始化已知数据集并进行存储 2. 对待分配的植物种类进行存储

Step1: 定义相关变量

- 1: 定义布尔变量: **flag1, flag2, flag3, flag4** 为 **False**
- 2: 定义最大差值 **maxi** 为 1
- 3: 定义土地编号 **bit** 为 0

Step2: 对于待分配的每个种类的植物

- 4: **for** 每个待分配植物种类 **do**

Situation1: 植物编号为 1-15

- 5: **if** 植物编号在 1 到 15 之间 **then**
- 6: 获取植物的第一、二、三类需求面积 (x_1, x_2, x_3)
- 7: **if** 第一类需求面积 $x_1 > 0$ **then**
- 8: **for** 遍历所有第一类土地 **do**
- 9: 检查是否为连续种植
- 10: **for** 遍历去年该地块的植物 **do**
- 11: **if** 去年种植的植物和当前植物相同 **then**
- 12: 设置 **flag1** 为 **True**
- 13: **end if**
- 14: **end for**
- 15: 检查是否种植豆类植物
- 16: **for** 遍历去年和前年的豆类植物 **do**
- 17: **if** 发现豆类植物 **then**
- 18: 设置 **flag2** 或 **flag3** 为 **True**
- 19: **end if**
- 20: **end for**
- 21: **if** 不需要种植豆类植物 **then**
- 22: 检查当前植物是否为豆类
- 23: **if** 是豆类植物 **then**
- 24: 设置 **flag4** 为 **True**
- 25: **end if**
- 26: **end if**
- 27: **if** 当前地块符合 Worst-Fit 算法且剩余面积大于 0 **then**
- 28: 更新最大差值 **maxi** 和 **bit**
- 29: **end if**
- 30: **end for**
- 31: **end if**
- 32: **end if**
- 33: **end for**

Algorithm 2 植物种类分配的最坏适应算法（第 2 部分）

```
1: for 每个待分配植物种类 do
2:   if 找到合适的地块 bit then
3:     if  $x_1$  大于等于该地块剩余面积 then
4:       将植物种植到该地块, 减少需求面积为 0
5:     else
6:       部分种植并减少需求面积
7:     end if
8:   end if
9:   if 第二类需求面积  $x_2 > 0$  then
10:    同样的逻辑判断和种植操作 (类似第一类土地)
11:   end if
12:   if 第三类需求面积  $x_3 > 0$  then
13:    同样的逻辑判断和种植操作 (类似第一类土地)
14:   end if
15: end for
```

Situation2: 植物编号为 16

```
16: if 植物编号等于 16 (水稻) then
17:   获取第四类需求面积  $x_4$ 
18:   for 遍历所有第四类土地 do
19:     进行同样的判断和种植操作
20:   end for
21: end if
```

Situation3: 植物编号为 17-34

```
22: if 植物编号在 17 到 34 之间 then
23:   获取第四、五、六类需求面积 ( $x_4, x_5, x_6$ )
24:   按照同样的逻辑分别判断和处理每一类土地
25: end if
```

Situation4: 植物编号为 35-41

```
26: if 植物编号在 35 到 41 之间 then
27:   获取第四、五类需求面积 ( $x_4, x_5$ )
28:   按照同样的逻辑分别判断和处理每一类土地
29: end if
```

Step3: 得到最终解决方案

5.1.4 求解结果

针对问题一的第一小问，结合上述线性规划方程和植物种类分配的最坏适应算法即可得到 2024 至 2030 年农作物的最优种植方式，未来 7 年具体种植方案在“result1_1”中，下面我们选取 2024 与 2027 的数据与 2023 年的数据对比得到不同农作物在各类型土地种植面积分布情况，代表性呈现我们的种植结果。

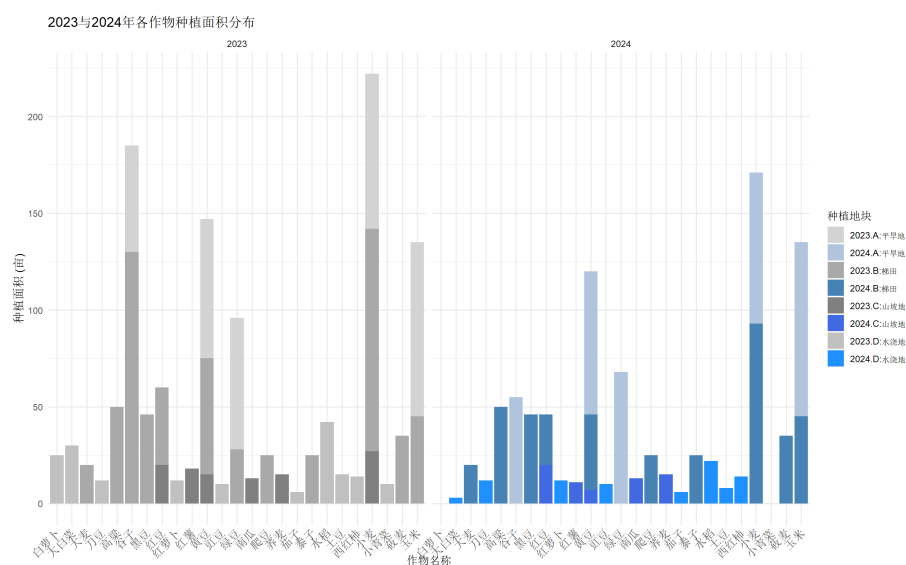


图 7 2024 年农作物种植分配情况对比-第一小问

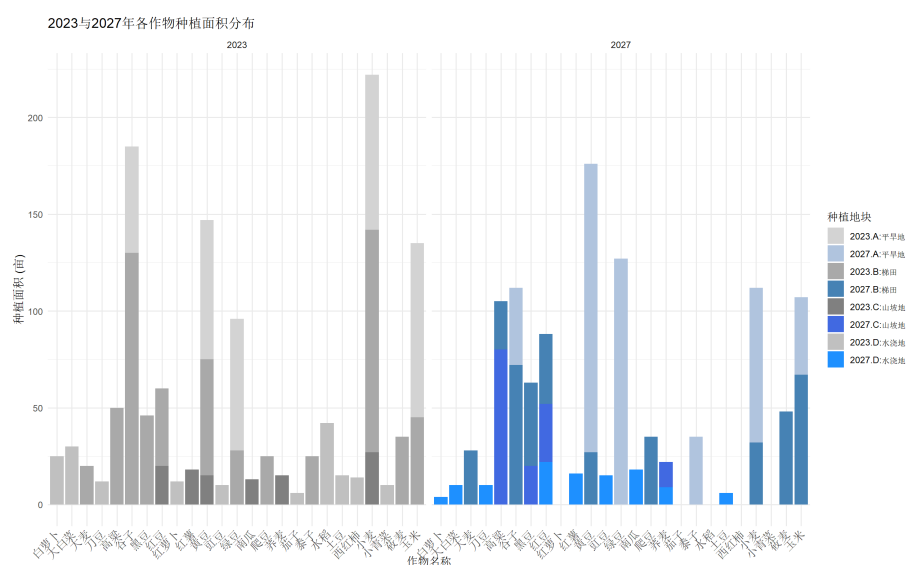


图 8 2027 年农作物种植分配情况对比-第一小问

	A	B	C	D	E	F	G	H	I	J	K	L	M	N	O	P	Q	R	S	T	U	V	W	X	Y	Z	AA	AB	AC	AD	AE
1	播种面积	玉米	大豆	花生	小麦	水稻	油菜	棉花	蔬菜	水果	其他	林地	草地	水域	未利用	其他	其他	其他	其他	其他	其他	其他	其他	其他	其他	其他	其他	其他	其他	其他	其他
2	2023	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	
3	2024	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	
4	2025	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	
5	2026	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	
6	2027	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	
7	2028	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	
8	2029	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	
9	2030	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	
10	2031	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	
11	2032	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	
12	2033	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	
13	2034	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	
14	2035	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	
15	2036	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	
16	2037	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	
17	2038	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	
18	2039	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	
19	2040	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	
20	2041	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	
21	2042	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	
22	2043	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	
23	2044	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	
24	2045	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	
25	2046	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	
26	2047	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	
27	2048	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	
28	2049	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	
29	2050	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	
30	2051	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	
31	2052	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	
32	2053	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	
33	2054	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	
34	2055	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	
35	2056	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	
36	2057	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	
37	2058	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	
38	2059	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	
39	2060	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	
40	2061	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	
41	2062	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	
42	2063	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	
43	2064	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	
44	2065	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	
45	2066	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	
46	2067	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	
47	2068	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	
48	2069	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	
49	2070	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	
50	2071	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	
51	2072	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	
52	2073	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	
53	2074	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	
54	2075	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	

图9 2027年 excel 表格部分截图-第一小问

针对问题一的第二小问,在问题一第一小问的约束条件下,我们已经找到了2024年到2030年农作物的最优种植方案。而通过上述图片可以观察得出土地面积并未得到完全利用。因此我们只需要在第一小问的基础上,找出未种植农作物的土地上所能种植的农作物中单位经济效益最高的农作物种植即可。按照求解问题一第一小问的方法对数据进行处理,建立线性规划模型,通过植物种类分配的最坏适应算法,即可得到结果。下面我们选取2024与2027的数据与2023年的数据对比得到不同农作物在各类型土地种植面积分布情况,代表性呈现我们的种植结果。

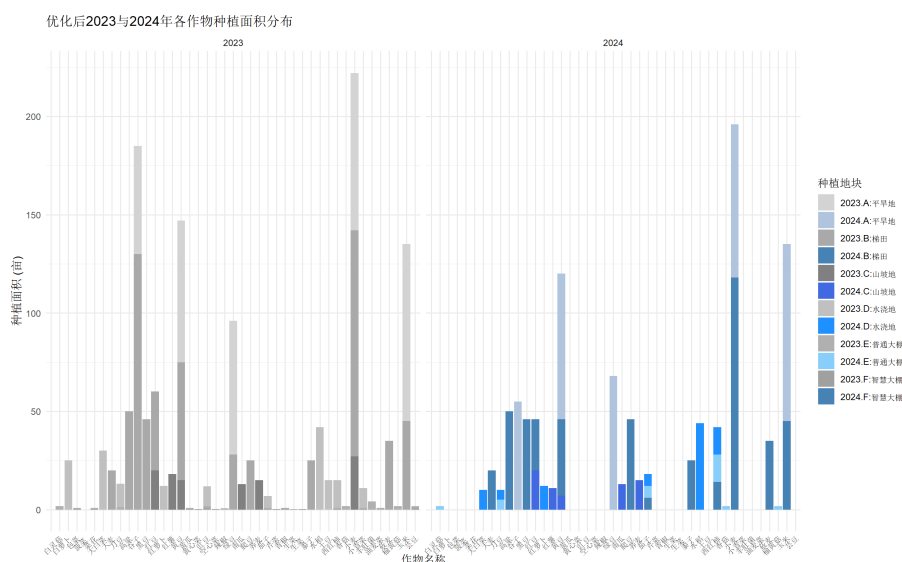


图10 2024年农作物种植分配情况对比-第二小问

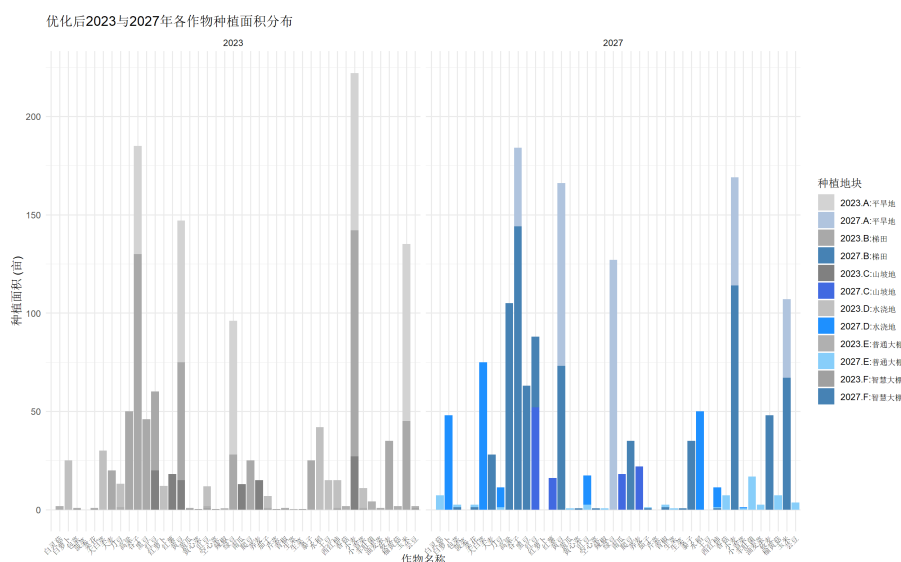


图 11 2027 年农作物种植分配情况对比-第二小问

	A	B	C	D	E	F	G	H	I	J	K	L	M	N	O	P	Q	R	S	T	U	V	W	X	Y	Z	AA	AB
1	地块名称	面积	玉米	大豆	花生	小麦	水稻	油菜	棉花	蔬菜	水果	其他	其他	其他	其他	其他	其他	其他	其他	其他	其他	其他	其他	其他	其他	其他	其他	其他
2	第 1 年	A1	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
3		A2	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
4		A3	35	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
5		A4	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
6		A5	68	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
7		A6	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
8		B1	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
9		B2	46	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
10		B3	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
11		B4	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
12		B5	0	24	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
13		B6	0	18	36	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
14		B7	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
15		B8	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
16		B9	27	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
17		B10	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
18		B11	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
19		B12	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
20		B13	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
21		B14	0	20	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
22		C1	0	0	15	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
23		C2	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
24		C3	0	0	15	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
25		C4	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
26		C5	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
27		C6	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
28		D1	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
29		D2	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
30		D3	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
31		D4	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
32		D5	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
33		D6	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
34		D7	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
35		D8	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
36		E1	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
37		E2	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
38		E3	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
39		E4	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
40		E5	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
41		E6	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
42		E7	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
43		E8	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
44		E9	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
45		E10	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
46		E11	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
47		E12	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
48		E13	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
49		E14	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
50		E15	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
51		E16	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
52		F1	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
53		F2	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0

图 12 2027 年 excel 表格部分截图-第二小问

5.2 问题二模型的建立及求解

5.2.1 模型准备：随机森林模型

随机森林模型是一种集成机器学习方法，通过多个决策树的组合来提高准确性和鲁棒性，常被用来解决较复杂的主控因子分析类问题，在该问题中，亩产量、复种指数、种植成本、单价等均可显著影响土地的经济效益，然而，其重要性排行尚不清晰，因此本文利用随机森林模型识别经济效益的关键性因子。

随机森林的核心思想为 Bagging（Bootstrap Aggregating）和随机特征选择。[4]

1.Bagging

其中, **Bagging** 是随机森林中最基础的技术, 用于在不增加偏差的情况下, 通过降低方差来提高模型的预测准确性和稳定性, 增强模型的泛化能力, 并且处理噪声和异常值的数据。通过 **Bootstrap** 抽样法, 从原始训练集 $D = \{(x_1, y_1), (x_2, y_2), \dots, (x_N, y_N)\}$ 中有放回地随机抽取出多个子集 D_t , 每个子集用于训练一个决策树 $h_t(x)$ 。具体公式如下:

$$D_t = \text{BootstrapSample}(D)$$

其中 t 表示第 t 个子集, $t = 1, 2, 3, \dots, T, T$ 为决策树的总数。[5]

2. 随机特征选择

在决策树构建过程中, 为增强随机性和模型的多样性, 在构建每棵树时, 随机选择一部分特征来决定每个节点的分裂。假设在 M 个特征中随意抽取 m 个。决策树的构建过程可以表示为:

Select m features from $M : \{f_1, f_2, \dots, f_m\} \subseteq \{F_1, F_2, \dots, F_M\}$

Split the node based on the best feature in $\{f_1, f_2, \dots, f_m\}$

其中, 回归问题常采用平均算法:

$$\hat{y} = \frac{1}{T} \sum_{t=1}^T h_t(x)$$

其中, T 为决策树的总数, $h_t(x)$ 为第 t 棵决策树对输入 x 的预测结果, 随机森林模型的优势在于其对噪声数据和异常值具有较强的鲁棒性, 且能够处理大量的输入变量, 而无需事先进行变量筛选。

此外, 通过内置的 *OOB* 误差估计, 随机森林能够在训练过程中自动估计模型的泛化误差, 从而提供对模型性能的一个有效评估。*OOB* 误差估计的公式如下:

$$\text{Error} = \frac{1}{N} \sum_{i=1}^N L(y_i, \hat{y}_i^{\text{OOB}})$$

其中, $\hat{y}_i^{\text{OOB}}$ 是第 i 个样本使用所有未包含该样本的树的预测结果, $L(y_i, \hat{y}_i)$ 是损失函数。

5.2.2 模型解答

1.2023 年各影响因子重要性排行

基于随机森林模型及 2023 年各因子数据分析可得, 输入的四个自变量因子销售单价、种植成本、亩产量和种植面积均通过显著性检验 ($p < 0.05$), 证明该四种变量均会显著影响总经济效益。在众多影响因子中, 亩产量与销售单价对总经济效益的影响贡献率最大, 其解释率分别达 31.30% 和 30.44%, 且二者均与单位面积经济效益呈现显著正相关关系, 故此可知, 在后续优化种植过程中应率先考虑销售单价高且亩产量较高的作物。

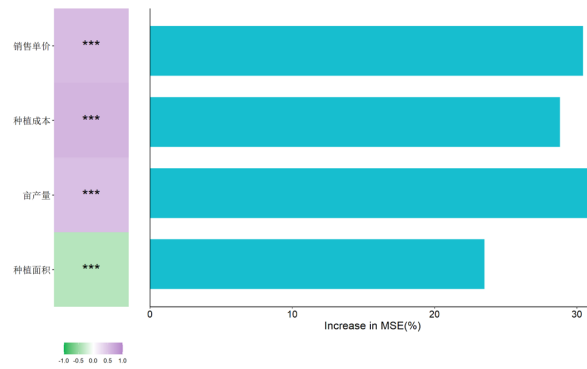


图 13 2023 年各影响因子重要性排行

2. 各影响因子波动对总经济效益的影响

基于随机森林模型对各因子变化值的预测分析发现，在最不利于总经济效益增加的情况下（即预期销售量、亩产量、销售价格达最大限度增长，种植成本平均达最低），亩产量的变化对总经济效益波动的影响最大，达 41.92%。而在最不利于总经济效益增加的情况下（即预期销售量、亩产量、销售价格达最大限度增长下降，种植成本平均达最高），销售单价的波动对总经济效益波动的影响最大，解释率达 36.29%。因此，在给出最佳种植方案时应主要考虑销售单价与亩产量的影响。

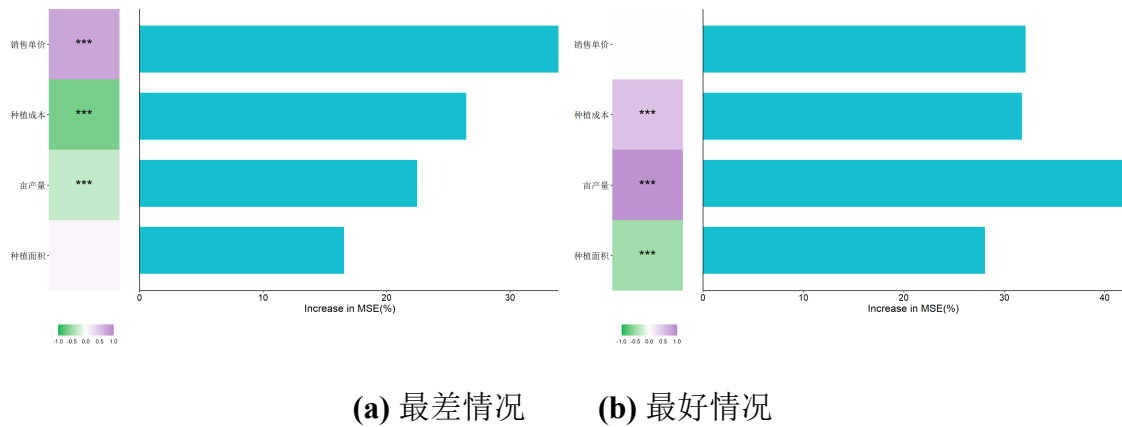


图 14 各影响因子波动对总经济效益的影响

5.2.3 求解结果

结合上述随机森林的分析结果可知，销售单价对最终总经济效益的影响最大，而在农作物未来的预期销售量、种植成本、亩产量和销售价格都有波动的情况下亩产量的变化对总经济效益波动的影响最大；因此在考虑最佳种植方案时，我们优先考虑销售单价和亩产量的变化。

而由问题一中的线性规划模型 $Z = CX$ ，其中的销售单价和亩产量对应变化影响的是目标函数中的价值向量 c_{ij} ，因此我们通过更新线性规划函数中的 C 经济效益矩阵，构

建出销售单价和亩产量的波动后的经济效益矩阵 C' , 得到新的线性规划函数 $Z' = C'X$, 得到的每种农作物在对应类型土地种植的总亩数, 最后通过植物种植最坏适应算法得到种植策略。

下面我们选取 2024 与 2027 的数据与 2023 年的数据对比得到不同农作物在各类型土地种植面积分布情况, 代表性呈现我们的种植结果。

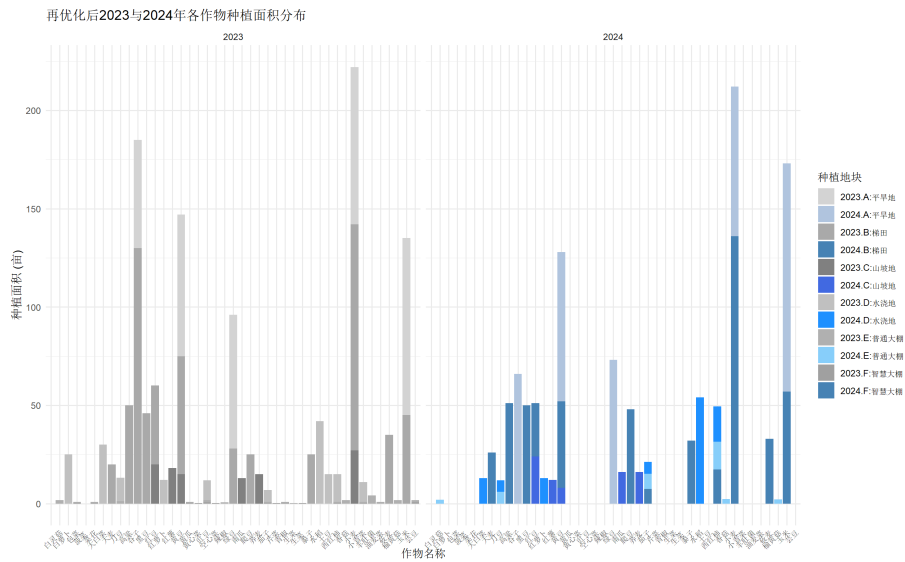


图 15 2024 年农作物种植分配情况对比-问题二

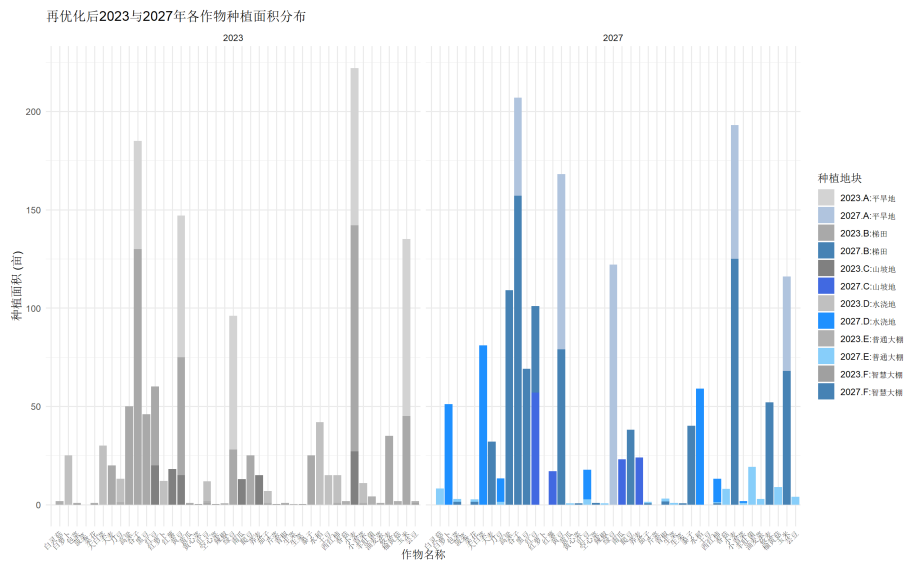


图 16 2027 年农作物种植分配情况对比-问题二

5.3 问题三模型的建立及求解

本文欲将理论与实际相结合, 在实现经济效益最大化的同时保障耕地国家安全, 探索出经济、生态、社会可持续发展的绿色道路。因此需要考虑较多生产实际问题, 本文

从作物间可替代性、不同作物间的相互影响、农作物对土地的改良、生产与销售间的供需关系及我国的国家政策等方面综合考虑并优化农作物耕作方案。

5.3.1 作物间的可替代性

文中所涉及的农作物可被进一步划分为豆类作物、谷类作物、蔬菜类作物、根茎作物、菌类作物等，其中，每类作物所提供的营养物质相似，如，豆类以提供蛋白质为主，谷类以提供糖类为主，蔬菜类以提供维生素为主。故此，当部分农作物经济效益较低而口感、营养价值等与其他种类农作物相似时，可考虑提换该种农作物。具体举例，大白菜和小青菜均可提供较丰富的维生素、矿物质和膳食纤维且均具有较低的热量，而由图 4，小青菜的单位面积经济价值远高于大白菜，二者种植环境相似，均为水浇地。因此，在生产实际中可以考虑提高小青菜种植面积。

5.3.2 不同作物间的相互影响

在具体种植过程中，还要具体考虑到田间管理与农业技术，通过农作物间的相互作用提高土地利用效率。在此过程中，可详细考虑同种土地类型各农作物的生态位，以充分利用作物生长空间，减少竞争。例如，同在水浇地种植，白萝卜做为根菜类作物可和小青菜等叶菜作物共同种植，以充分利用太阳能及土地等资源，提高总产量。

其次，也可考虑到农作物对局地小气候的改善作用，如，同在普通大棚中，香菇的生长需较为湿润的环境，而油麦菜等叶菜类作物可通过遮荫效果等减少水分蒸发，因而使局地小气候较为湿润，利于香菇单位面积产量及品质的增长，从而提高总经济效益。

5.3.3 农作物对土地的改良

某作物的根系可改善耕地质量，因而可因地制宜种植作物以改良土壤。例如，在缺氮条件下的土壤可多种植豆类植物，例如大豆根系可与根瘤菌共生，从而起到固氮效果，为周围农作物提供氮源，提高土壤肥力，增加亩产量。再如，白萝卜等根系可分泌有机酸，从而溶解土壤中矿物质，便于其他农作物吸收以提高产量。

研究区域位于华北山区，部分土地可能发生水土流失现象，故此可考虑在平旱地及梯田等地块种植红薯、玉米、水稻等深根系作物，以产生经济收益的同时保护生态环境，减少水土流失。

5.3.4 生产与销售间的供需关系

由图 15 可知，农作物生产与销售间满足供需关系，农作物的生产量直接影响市场供应：如果生产过剩，可能会导致价格下降；如果生产不足，则可能导致价格上涨。而消费者需求的变化会影响市场销售：需求增加可能促使生产者增加生产，而需求减少可

能导致生产减少。因此可基于前些年市场销售数据，进行 Mann-Kendall 检验，以分析和预测市场销售量，根据市场销售量匹配种植相应农作物。

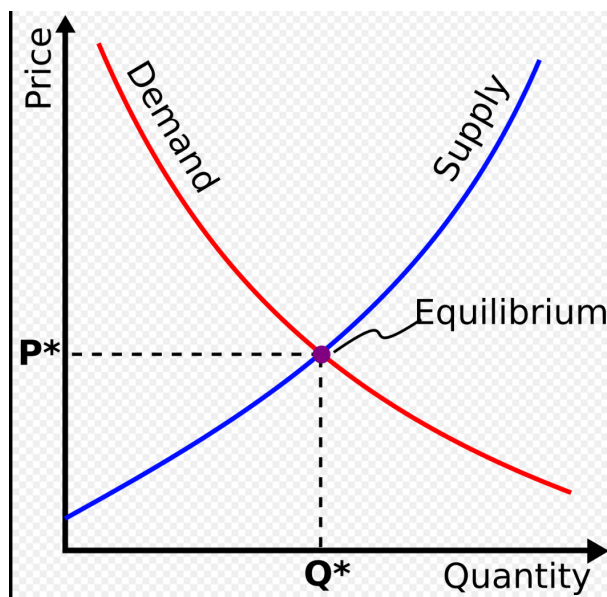


图 17 供需关系 [6]

5.3.5 我国的国家政策

近年来我国提出坚守“十八亿亩耕地红线”政策，即“牢牢将饭碗掌握在我们自己手中”，着重强调保护粮食国家安全。我国人口稠密，人均耕地资源不足，因此在考虑经济价值的同时应首先保证粮食作物的种植面积，将较大块耕地分配给粮食作物。

六、模型的评价

6.1 模型的优点

1. 进行了数据预处理，可视化数据便于理解和推广，期间得到的中间数据具有较高的准确率。
2. 对于问题的具体解决方案，结合了数学和计算机领域的不同算法，体现了学科交叉的思想，做到了应用理论到具体实际问题。
3. 采用基础的线性规划思维，思路清晰简单。
3. 未知量设置的较为合理，最后得到的结果较为简便且易于理解。

6.2 模型的缺点

1. 对于第二问的多种影响因子考虑并不完全，只考虑了一部分的影响。对于特殊情况的判断并未全部考虑到，会有部分错漏。

2. 算法思想过于复杂，不够简便。
3. 最坏适应模型虽然算法试图避免碎片化，但在某些情况下，仍然可能导致内存的分散和浪费，尤其是在任务内存需求大小不一且频繁变化的情况下。

七、模型的推广

本文中模型的应用背景是作物在多种条件下的规划种植问题，目的是为了取得最大的收益。在相似背景下的运用还有很多，例如涉及到空间分配以及获取收益的问题。

参考文献

- [1] 钟迎珍. 乡村振兴战略背景下我国粮食安全面临的挑战及对策研究 [N]. 河南经济报,2024-07-06(009).
- [2] 姜启源, 谢金星, 叶俊. 数学模型 (第四版) [M]. 北京: 高等教育出版社, 2011.
- [3] Amudha, A., and C. Christofer Asir Rajan. "Effect of reserve in profit based Unit Commitment using Worst Fit algorithm." 2011 International Conference on Process Automation, Control and Computing. IEEE, 2011.
- [4] Lin, Weiwei, et al. "An ensemble random forest algorithm for insurance big data analysis." Ieee access 5 (2017): 16568-16575
- [5] 秦志军, 操应长, 冯程. 基于改进型随机森林算法的页岩岩性识别——以准噶尔盆地芦草沟组为例 [J/OL]. 新疆石油地质,1-9[2024-09-08].
- [6] SilverStar. 供给-需求-平衡图片.
<https://commons.wikimedia.org/w/index.php?curid=4681584>

附录 A 文件支撑目录

源.cpp
output1_2024.txt
output1_2025.txt
output1_2026.txt
output1_2027.txt
output1_2028.txt
output1_2029.txt
output1_2030.txt
预计产量表.xlsx
result1_1.xlsx
result1_2.xlsx
result2.xlsx
excel_处理.py
Linprog.py
matlab_LP_Question.py
可视化图解.py

附录 B 模型计算结果

附件部分截图

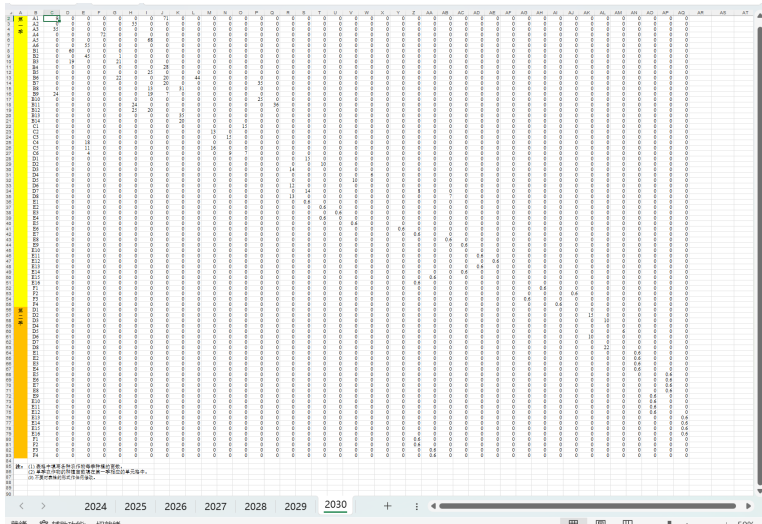


图 18 问题一第一小问部分截图

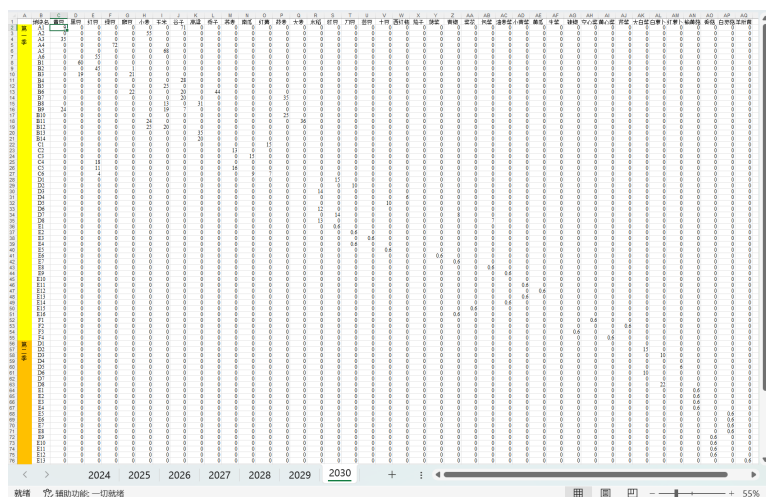


图 19 问题一第二小问部分截图



图 20 问题二部分截图

计算结果

平旱地：

A1 80 谷子 71 黄豆 9

A2 55 小麦 55

A3 35 黄豆 35

A4 72 绿豆 72

A5 68 玉米 68

A6 55 绿豆 55

梯田：

B1 60 黑豆 60

B2 46 红豆 45 爬豆 1

B3 40 黑豆 19 爬豆 21

B4 28 谷子 28
B5 25 玉米 25
B6 86 黍子 44 爬豆 22 谷子 20
B7 55 莠麦 35 谷子 20
B8 44 高粱 31 玉米 13
B9 50 黄豆 24 玉米 19 谷子 7
B10 25 莠麦 25
B11 60 大麦 36 小麦 24
B12 45 小麦 25 玉米 20
B13 35 高粱 35
B14 20 高粱 20

山坡地:

C1 15 红薯 15
C2 13 荞麦 13
C3 15 南瓜 15
C4 18 红豆 18
C5 27 荞麦 16 红豆 11
C6 20 南瓜 9 红薯 7 红豆 4

水浇地:

D1 15 白萝卜 9 红萝卜 6
D2 10 刀豆 10
D3 14 西红柿 14
D4 6 水稻 6
D5 10 豇豆 10
D6 12 土豆 12
D7 22 茄子 8 豇豆 14
D8 20 大白菜 7 水稻 13

普通大棚:

E1 0.6 羊肚菌 0.6
E2 0.6 白灵菇 0.6
E3 0.6 香菇 0.6
E4 0.6 榆黄菇 0.6
E5 0.6 茄子 0.6
E6 0.6 西红柿 0.6
E7 0.6 芸豆 0.6
E8 0.6 刀豆 0.6
E9 0.6 辣椒 0.6
E10 0.6 黄瓜 0.6
E11 0.6 小青菜 0.6
E12 0.6 油麦菜 0.6
E13 0.6 包菜 0.6
E14 0.6 菜花 0.6
E15 0.6 青椒 0.6
E16 0.6 芸豆 0.6

智慧大棚:

F1 0.6 芹菜 0.6
F2 0.6 黄心菜 0.6

F3 0.6 空心菜 0.6

F4 0.6 生菜 0.6

附录 C 模型计算代码

Linprog 函数

```
1. import pandas as pd
2. import numpy as np
3. from scipy.optimize import linprog
4.
5. # 读取Excel文件
6. file_path = r"C:\Users\31105\Desktop\new_matlab.xlsx"
7.
8. # 读取目标函数的系数c (123个未知数的参数) 从sheet1
9. df_sheet1 = pd.read_excel(file_path, sheet_name='sheet1')
10. c_neg = df_sheet1['123个未知数的参数'].values # 目标函数的系数
11. print("目标函数的系数 (c_neg) :")
12. print(c_neg)
13.
14. c = -c_neg
15. print("目标函数的系数 (c) :")
16. print(c)
17.
18. # 读取约束条件矩阵A从sheet2
19. df_sheet2 = pd.read_excel(file_path, sheet_name='sheet2')
20. print("约束条件矩阵 (df_sheet2) :")
21. print(df_sheet2)
22.
23. print("df_sheet2的形状:")
24. print(df_sheet2.shape)
25.
26. # 初始化不等式约束矩阵 A_ub (41x123)
27. A_ub = np.zeros((41, 123))
28.
29. # 填充约束条件矩阵
30. for i in range(min(41, df_sheet2.shape[0])):
31.     row = df_sheet2.iloc[i, :3].values # 当前行的系数
32.     indices = [i * 3, i * 3 + 1, i * 3 + 2] # 当前行对应的未知数的索引
33.     A_ub[i, indices] = row # 将系数填入约束矩阵
34.
35. print("初始不等式约束矩阵 A_ub:")
36. print(A_ub.shape)
37.
```

```

38. # 读取不等式约束的右侧数值b_ub (41个总产量约束系数) 从sheet1
39. b_ub = df_sheet1['41个总产量约束系数'].values[:41] # 不等式约束右侧的数值
40. print("不等式约束右侧的数值 b_ub:")
41. print(b_ub)
42.
43.
44. # 添加额外的不等式约束
45. # 1. 第1-15行第一列数据对应的未知数之和要小于365
46. A_ub = np.vstack([A_ub, np.zeros(123)])
47. A_ub[-1, np.arange(0, 43, 3)] = 1 # 对应第1-15行的第一列
48. b_ub = np.append(b_ub, 365)
49.
50. # 2. 第1-15行第二列数据对应的未知数之和要小于619
51. A_ub = np.vstack([A_ub, np.zeros(123)])
52. A_ub[-1, np.arange(1, 44, 3)] = 1 # 对应第1-15行的第二列
53. b_ub = np.append(b_ub, 619)
54.
55. # 3. 第1-15行第三列数据对应的未知数之和要小于108
56. A_ub = np.vstack([A_ub, np.zeros(123)])
57. A_ub[-1, np.arange(2, 45, 3)] = 1 # 对应第1-15行的第三列
58. b_ub = np.append(b_ub, 108)
59.
60. # 4. 第16行第一列数据 + 第17-34行第一列数据 +
    第35-37行第一列数据对应的未知数的和要小于109
61. A_ub = np.vstack([A_ub, np.zeros(123)])
62. A_ub[-1, [45] + np.arange(48, 100, 3).tolist() + np.arange(104, 111).tolist()] = 1
    # 对应第16、17-34、35-37行第一列
63. b_ub = np.append(b_ub, 109)
64.
65. # 5. 第17-34行第二列数据 + 第38-41行第一列数据对应的未知数的和要小于9.6
66. A_ub = np.vstack([A_ub, np.zeros(123)])
67. A_ub[-1, np.arange(49, 101, 3).tolist() + [111, 114, 117, 120]] = 1
68. b_ub = np.append(b_ub, 9.6)
69.
70. # 6. 第17-34行第三列数据对应的未知数的和要小于2.4
71. A_ub = np.vstack([A_ub, np.zeros(123)])
72. A_ub[-1, np.arange(50, 102, 3).tolist()] = 1
73. b_ub = np.append(b_ub, 2.4)
74.
75.
76. print("添加约束后的不等式约束矩阵 A_ub:")
77. print(A_ub)
78.
79. print("添加约束后的不等式约束右侧的数值 b_ub:")
80. print(b_ub)
81.
82. print("A_ub的形状:")

```

```

83. print(A_ub.shape)
84. print("b_ub的形状:")
85. print(b_ub.shape)
86.
87. # 求解线性规划问题
88. res = linprog(c, A_ub=A_ub, b_ub=b_ub, bounds=[(0, None)] * 123, method='highs')
89.
90. print("优化结果: ", res)
91.

```

最坏适应算法-部分

```

1. void WorstFit(int nowYear, int num)
2. {
3.     bool flag1 = false, flag2 = false, flag3 = false, flag4 = false;
4.     double maxi = -1;
5.     int bit = 0;
6.
7.     //分为四大类植物
8.     if (num >= 1 && num <= 15) {
9.         double x1, x2, x3;
10.        x1 = plant[num].farea[1];
11.        x2 = plant[num].farea[2];
12.        x3 = plant[num].farea[3];
13.
14.        if (x1) {
15.            //先判断第一类土地所满足的条件
16.            for (int i = 1; i <= 6; i++) {
17.                //先判断是否连续种植
18.                for (int j = 1; year[nowYear].land1[i].fname[j] != ""; j++) {
19.                    string nowname = plant[num].name;
20.                    for (auto s : year[nowYear - 1].land1[i].fname) {
21.                        if (s == nowname) {
22.                            flag1 = true;
23.                            break;
24.                        }
25.                    }
26.                }
27.                //再判断是否种豆
28.                for (auto x : sss) {
29.                    for (int j = 1; year[nowYear - 1].land1[i].fname[j] != ""; j++) {
30.                        string nowname = year[nowYear - 1].land1[i].fname[j];
31.                        if (x == nowname) flag2 = true;
32.                        break;
33.                    }

```

```

34.     }
35.     for (auto x : sss) {
36.         for (int j = 1; year[nowYear - 2].land1[i].fname[j] != ""; j++) {
37.             string nowname = year[nowYear - 2].land1[i].fname[j];
38.             if (x == nowname)flag3 = true;
39.             break;
40.         }
41.     }
42.     //如果需要进行种豆
43.     if (!(flag2 || flag3)) {
44.         //再判断当前植物是不是豆类
45.         string nowname = plant[num].name;
46.         for (auto x : sss) {
47.             if (x == nowname) {
48.                 flag4 = true;
49.                 break;
50.             }
51.         }
52.     }
53.     if (flag4 && year[nowYear].land1[i].larea > 0) {
54.         bit = i;
55.         break;
56.     }
57.
58.     //再判断是否满足WF算法
59.     if (fabs(x1 - year[nowYear].land1[i].larea) >= maxi &&
        year[nowYear].land1[i].larea > 0) {
60.         maxi = fabs(x1 - year[nowYear].land1[i].larea);
61.         bit = i;
62.     }
63. }
64. if (bit) {
65.     if (x1 >= year[nowYear].land1[bit].larea) {
66.         year[nowYear].land1[bit].cnt++;
67.         year[nowYear].land1[bit].fname[year[nowYear].land1[bit].cnt] = plant[num].name;
68.         year[nowYear].land1[bit].farea[year[nowYear].land1[bit].cnt] =
            year[nowYear].land1[bit].larea;
69.         plant[num].farea[1] -=
            year[nowYear].land1[bit].farea[year[nowYear].land1[bit].cnt];
70.         year[nowYear].land1[bit].farea[year[nowYear].land1[bit].cnt] =
            year[nowYear].land1[bit].larea;
71.         year[nowYear].land1[bit].larea = 0;
72.         return;
73.     }
74.     else {
75.         year[nowYear].land1[bit].cnt++;
76.         year[nowYear].land1[bit].fname[year[nowYear].land1[bit].cnt] = plant[num].name;

```



```

77.     year[nowYear].land1[bit].farea[year[nowYear].land1[bit].cnt] = x1;
78.     year[nowYear].land1[bit].larea -= plant[num].farea[1];
79.     plant[num].farea[1] = 0;
80.
81.     return;
82. }
83. }
84. }
85.
86. //再进行第二类的判断
87. if (x2) {
88.     for (int i = 1; i <= 14; i++) {
89.         //先判断是否连续种植
90.         for (int j = 1; year[nowYear].land2[i].fname[j] != ""; j++) {
91.             string nowname = plant[num].name;
92.             for (auto s : year[nowYear - 1].land2[i].fname) {
93.                 if (s == nowname) {
94.                     flag1 = true;
95.                     break;
96.                 }
97.             }
98.         }
99.         //再判断是否种豆
100.        for (auto x : sss) {
101.            for (int j = 1; year[nowYear - 1].land2[i].fname[j] != ""; j++) {
102.                string nowname = year[nowYear - 1].land2[i].fname[j];
103.                if (x == nowname) flag2 = true;
104.                break;
105.            }
106.        }
107.        for (auto x : sss) {
108.            for (int j = 1; year[nowYear - 2].land2[i].fname[j] != ""; j++) {
109.                string nowname = year[nowYear - 2].land2[i].fname[j];
110.                if (x == nowname) flag3 = true;
111.                break;
112.            }
113.        }
114.        //如果需要进行种豆
115.        if (!(flag2 || flag3)) {
116.            //再判断当前植物是不是豆类
117.            string nowname = plant[num].name;
118.            for (auto x : sss) {
119.                if (x == nowname) {
120.                    flag4 = true;
121.                    break;
122.                }
123.            }

```

```

124.     }
125.     if (flag4 && year[nowYear].land2[i].larea > 0) {
126.         bit = i;
127.         break;
128.     }
129.
130.     //再判断是否满足WF算法
131.     if (fabs(x2 - year[nowYear].land2[i].larea) >= maxi &&
        year[nowYear].land2[i].larea > 0) {
132.         maxi = fabs(x2 - year[nowYear].land2[i].larea);
133.         bit = i;
134.     }
135. }
136. if (bit) {
137.     if (x2 >= year[nowYear].land2[bit].larea) {
138.         year[nowYear].land2[bit].cnt++;
139.         year[nowYear].land2[bit].fname[year[nowYear].land2[bit].cnt] = plant[num].name;
140.         year[nowYear].land2[bit].farea[year[nowYear].land2[bit].cnt] =
            year[nowYear].land2[bit].larea;
141.         plant[num].farea[2] -=
            year[nowYear].land2[bit].farea[year[nowYear].land2[bit].cnt];
142.         year[nowYear].land2[bit].farea[year[nowYear].land2[bit].cnt] =
            year[nowYear].land2[bit].larea;
143.         year[nowYear].land2[bit].larea = 0;
144.         return;
145.     }
146.     else {
147.         year[nowYear].land2[bit].cnt++;
148.         year[nowYear].land2[bit].fname[year[nowYear].land2[bit].cnt] = plant[num].name;
149.         year[nowYear].land2[bit].farea[year[nowYear].land2[bit].cnt] = x2;
150.         year[nowYear].land2[bit].larea -= plant[num].farea[2];
151.         plant[num].farea[2] = 0;
152.
153.         return;
154.     }
155. }
156. }
157.
158. //再进行第三类的判断
159. if (x3) {
160.     for (int i = 1; i <= 6; i++) {
161.         //先判断是否连续种植
162.         for (int j = 1; year[nowYear].land3[i].fname[j] != ""; j++) {
163.             string nowname = plant[num].name;
164.             for (auto s : year[nowYear - 1].land3[i].fname) {
165.                 if (s == nowname) {
166.                     flag1 = true;

```

```

167.         break;
168.     }
169. }
170. }
171. //再判断是否种豆
172. for (auto x : sss) {
173.     for (int j = 1; year[nowYear - 1].land3[i].fname[j] != ""; j++) {
174.         string nowname = year[nowYear - 1].land3[i].fname[j];
175.         if (x == nowname)flag2 = true;
176.         break;
177.     }
178. }
179. for (auto x : sss) {
180.     for (int j = 1; year[nowYear - 2].land3[i].fname[j] != ""; j++) {
181.         string nowname = year[nowYear - 2].land3[i].fname[j];
182.         if (x == nowname)flag3 = true;
183.         break;
184.     }
185. }
186. //如果需要进行种豆
187. if (!(flag2 || flag3)) {
188.     //再判断当前植物是不是豆类
189.     string nowname = plant[num].name;
190.     for (auto x : sss) {
191.         if (x == nowname) {
192.             flag4 = true;
193.             break;
194.         }
195.     }
196. }
197. if (flag4 && year[nowYear].land3[i].larea > 0) {
198.     bit = i;
199.     break;
200. }
201.
202. //再判断是否满足WF算法
203. if (fabs(x3 - year[nowYear].land3[i].larea) >= maxi &&
    year[nowYear].land3[i].larea > 0) {
204.     maxi = fabs(x3 - year[nowYear].land3[i].larea);
205.     bit = i;
206. }
207. }
208. if (bit) {
209.     if (x3 >= year[nowYear].land3[bit].larea) {
210.         year[nowYear].land3[bit].cnt++;
211.         year[nowYear].land3[bit].fname[year[nowYear].land3[bit].cnt] = plant[num].name;
212.         year[nowYear].land3[bit].farea[year[nowYear].land3[bit].cnt] =

```

```

        year[nowYear].land3[bit].larea;
213.     plant[num].farea[3] -=
        year[nowYear].land3[bit].farea[year[nowYear].land3[bit].cnt];
214.     year[nowYear].land3[bit].farea[year[nowYear].land3[bit].cnt] =
        year[nowYear].land3[bit].larea;
215.     year[nowYear].land3[bit].larea = 0;
216.     return;
217. }
218. else {
219.     year[nowYear].land3[bit].cnt++;
220.     year[nowYear].land3[bit].fname[year[nowYear].land3[bit].cnt] = plant[num].name;
221.     year[nowYear].land3[bit].farea[year[nowYear].land3[bit].cnt] = x3;
222.     year[nowYear].land3[bit].larea -= plant[num].farea[3];
223.     plant[num].farea[3] = 0;
224.
225.     return;
226. }
227. }
228. }
229. }
230. //水稻与水浇地的情况
231. else if (num == 16) {
232.     double x4;
233.     x4 = plant[num].farea[4];
234.
235.     for (int i = 1; i <= 8; i++) {
236.         //先判断是否为连续种植
237.         for (int j = 1; year[nowYear].land4[i].fname[j] != ""; j++) {
238.             string nowname = plant[num].name;
239.             for (auto s : year[nowYear - 1].land4[i].fname) {
240.                 if (s == nowname) {
241.                     flag1 = true;
242.                     break;
243.                 }
244.             }
245.         }
246.
247.         //再判断是否需要种豆
248.         for (auto x : sss) {
249.             for (int j = 1; year[nowYear - 1].land4[i].fname[j] != ""; j++) {
250.                 string nowname = year[nowYear - 1].land4[i].fname[j];
251.                 if (x == nowname) flag2 = true;
252.                 break;
253.             }
254.         }
255.         for (auto x : sss) {
256.             for (int j = 1; year[nowYear - 2].land4[i].fname[j] != ""; j++) {

```

```

257.     string nowname = year[nowYear - 2].land4[i].fname[j];
258.     if (x == nowname)flag3 = true;
259.     break;
260. }
261. }
262. if (!(flag2 || flag3)) {
263.     //再判断当前植物是不是豆类
264.     string nowname = plant[num].name;
265.     for (auto x : sss) {
266.         if (x == nowname) {
267.             flag4 = true;
268.             break;
269.         }
270.     }
271. }
272. //对于需要种植的豆类蔬菜，我们选择进行特殊情况特殊判断
273. /*if (flag4) {
274.     bit = i;
275.     break;
276. }*/
277. //如果需要种植豆类，则该块土地暂时闲置，进行下一块土地的判断
278. if (!(flag2 || flag3) && !flag4) {
279.     continue;
280. }

```

数据可视化与处理-部分

```

95.
96. import pandas as pd
97. import re
98. """
99. # 读取Excel文件
100. file_path = r
101. df = pd.read_excel(file_path)
102.
103. # 处理“种植地块”列，只保留字母，去除数字
104. df['种植地块'] = df['种植地块'].apply(lambda x: ''.join(re.findall(r'[A-Za-z]',
    str(x))))
105.
106. # 保存修改后的结果到新Excel文件
107. output_file = r
108. df.to_excel(output_file, index=False)
109.
110. print("处理完成并保存到新文件。")
111.

```

```

112. """
113.
114. # 读取Excel文件
115. file_path = r"C:\Users\31105\Desktop\23年种植汇总.xlsx"
116. df = pd.read_excel(file_path)
117.
118. # 定义一个函数来计算两个数字的平均值
119. def calculate_average(price_range):
120.     try:
121.         # 将字符串按照 '-' 分割成两个数字
122.         low, high = price_range.split('-')
123.         # 计算平均值
124.         return (float(low) + float(high)) / 2
125.     except:
126.         return None # 处理异常情况
127.
128. # 计算“销售单价”列的平均值
129. df['平均销售单价'] = df['销售单价'].apply(calculate_average)
130.
131. # 将计算结果保存到新的sheet中
132. with pd.ExcelWriter(file_path, engine='openpyxl', mode='a') as writer:
133.     df[['销售单价', '平均销售单价']].to_excel(writer, sheet_name='price',
134.                                                index=False)
134.
135. print("计算完成并保存到新sheet。")

```